

MODULO 1

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

- 1) Python - “-4, 512”, a potência do 2 vai ter prioridade ao sinal de negativo porque não está entre parênteses e a potência de 2 elevado a 3 elevado a 2 será calculada corretamente
- 2) Java - “3 3 4 50”, a princípio soma 1 a 2, resultando 3, depois é concatenado a uma string, a partir desse momento o resto do print se torna uma string, concatenando 4 e então 5.
- 3) JavaScript - “3, 52”, quando for subtraído, a operação será executada corretamente, mas utilizando o sinal de soma vamos realizar uma operação de concatenação não proposital.
- 4) C - “1”, a impressão de um inteiro vai ler “3 > 2 > 1” como 1, por ser uma operação verdadeira
- 5) Python - “padrão, True”, na operação ‘x or “padrão”’, vamos imprimir “padrão”, e depois da vírgula, a operação “3 > 2 > 1” retorna True
- 6) JavaScript - “True, False, True”, a última operação enxerga ambos os zeros como iguais, as duas primeiras eu só sei por experiência prévia.

MODULO 2

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Precedência e ordem: depende bué da linguagem. No Python/JS 232 dá 512 porque lê da direita pra esquerda. Mas no Excel o menos vence a potência, mó confusão. Na dúvida pôr SEMPRE parênteses.

Divisão e Tipos: em Python — $7/2$ dá float (— 3.5), se quiser inteiro tem de ser $//$. Java arredonda pra — 3.

Efeitos colaterais (cuidado no exame!!): se uma função muda uma variável global tipo $a = a + fun1()$, o resultado muda dependendo do compilador. Em Java dá 8 (lê a da esquerda primeiro), em C/Go dá 20 (chama a função antes).

Sobrecarga de operadores: o $+$ serve pra somar ou juntar texto. Em Python dá pra redefinir isso com `__add__`. Java NÃO deixa criar novos, só aceita no $+$ nativo.

Pegadinha de Java: 'a' + 1 dá 98 (usa ASCII), mas "" + 'a' + 1 vira a string "a1".

MODULO 3

Conversões de tipos: alargamento, estreitamento e coerção

Java - coerção e cast

```
int grande = 123456789;
float f = grande; // alargamento implícito
System.out.println(int f); // 123456792

double d = 3.99;
System.out.println(int d); // 3 (cast)
System.out.println(byte 200); // -56

byte b = 10, c = 20;
// byte a = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem int → real (implícito) int32 + int64

C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não	*float64(1) erro de compilação
Rust	não	1 as f64 erro de compilação
ML, F# (p. 313)	não	erro de compilação

Coerção esconde erros (p. 312): em $d = b * a$; com a digitado por engano no lugar de c, o Java converte e compila.

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("d\n", (a > b) && (b > c)); // 1
if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sobota, 11ª ed., §7.5, p. 315-317, exemplos05, booknotes

Avaliação em curto-circuito

Java - && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++; // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript - ? e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco.cidade // undefined, sem erro

const qtd = 0;
qtd || 10 // 10 (o 0 se perdeu!)
qtd ?? 10 // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em $(a > b) || (b++ > 3)$, o $b++$ só acontece quando $a <= b$. Em Python, 0 or "padrão" devolve "padrão"; and e or devolvem um dos operandos.

Sobota, 11ª ed., §7.6, p. 315-317, exemplos05, booknotes

Conversões são ocasiões em que precisamos mudar o tipo da variável, comportando um número inteiro para um número real, ou o contrário, podendo perder ou não informações no processo. Um alargamento vai converter para um tipo “maior”, enquanto um estreitamento converte para um tipo que não comporta todo o valor de armazenamento do original. Coerção é uma conversão implícita, feita pelo próprio compilador. Casting é uma conversão explícita, pedida pelo programador. Operadores relacionais têm preferência menor que os aritméticos e seus resultados são booleanos. No exemplo acima, em C: “ $3 > 2 > 1$ ” é falso, porque primeiro vê $3 > 2$ como 1 (True) e só então, $1 > 1$ como 0 (False), já em Python, a execução é encadeada, e vê “ $3 > 2 > 1$ ” direto como True. Curto-circuito é quando uma operação é interrompida antecipadamente porque seu restante é considerado irrelevante, por exemplo: Se a primeira condição de um AND for falsa, já temos que a operação final vai ser falsa.

MODULO 4

Sentenças de atribuição

C - a atribuição é uma expressão (p. 319-320)

```
sum = count = 0; // múltiplos alvos
a = b + (c = d / b) - 1; // exemplo do livro
while ((ch = getchar()) != EOF) { ... }
if (x = y) { ... } // compilador atribui e testa y
```

Python - atribuição múltipla (p. 320)

```
a, b = b, a # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)
val (ML) e let (F#, Rust) dão nome a um valor, que não muda.

Atribuição como expressão: em C/C++/JS a atribuição devolve um valor, por isso if (x = y) compila e pode gerar bugs! Java/C# só deixam se o resultado for booleano. Python exige o operador morsa :=, enquanto Go e Rust bloqueiam totalmente.

Atribuição múltipla e linguagens funcionais:

Python permite trocar valores direto com a, b = b, a. Em linguagens funcionais usam-se termos

como val ou let para definir valores imutáveis. Mistura de tipos e conversões: Java só converte automaticamente para tipos maiores (de int para double), mas atenção ao i += 3.7, onde ele faz um cast escondido sem avisar! O C faz conversões de forma totalmente silenciosa (ex: int i = 3.7 vira 3).

MODULO 5

Estruturas básicas vs. Excesso: Teorema de Böhm e Jacopini (1966) provou que apenas sequência, seleção e iteração chegam para qualquer algoritmo (o goto é desnecessário). Outras estruturas (como for, switch, do-while) existem apenas para facilitar a escrita, mas podem prejudicar a legibilidade.

O problema do else pendente: Em C, Java e C++, o else liga-se sempre ao if mais próximo que ainda não tenha else, ignorando totalmente a indentação! Linguagens como Go, Rust e Perl evitam isto tornando as chaves {} obrigatórias. Seleção Múltipla (switch / match):

- Em C, se não colocares break, o código "cai" para o caso seguinte (fallthrough) e executa tudo.
- Java (14+) e Rust usam seleções que devolvem valor. O match do Rust obriga a cobrir todos os casos possíveis (exaustivo).

Seleção múltipla: do switch de C ao match

C - sem break, o continue "cai" (p. 335)	Java 14+ - switch como expressão	Rust - match exaustivo
<pre>switch (10000) { case 1: case 10: cont = 1; case 2: case 3: case 4: default: printf("erro!"); } // index = 1: ok, even é errado!</pre>	<pre>String tipo = switch (dia) { case 300, 301 -> "fim de semana"; case 300, 301, 302 -> "dia de trabalho"; case 303, 304 -> "dia de trabalho"; // omitindo, sem case! // exaustivo para enum } // index = 1: ok, even é errado!</pre>	<pre>match n { 123456789..-1 => "negativo", 0 => "zero", 1..9 => "um dígito", .. => "múltiplos dígitos", } // sem 0 .. -1 não compila</pre>

Go: cada caso termina sozinho, e car exige fallthrough. C#: car implicitamente é proibido (p. 336). Python 3.10: match com pattern e guards.

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)	JavaScript - for...in vs for...of
<pre>externo: for (int i = 0; i < m.length; i++) for (int j = 0; j < m[i].length; j++) if (m[i][j] < 0) { System.out.println(i + ", " + j); break externo; // sai dos DOIS laços }</pre>	<pre>const notas = [10, 20, 30]; let total = 0; for (const i in notas) total += i; console.log(total); // "0012": chaves! total = 0; for (const v of notas) total += v; console.log(total); // 60: valores</pre>

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa ao menos uma vez, e isso é fácil de esquecer (p. 347).

Laços de Repetição:

- Contadores: Em C, alterar a variável de controle dentro do for muda as iterações. Em Python, o for percorre um iterável (como um range) e alterar a variável do laço lá dentro não afeta a contagem.
 - Comandos de Saída: Java permite break rotulado para sair de múltiplos laços aninhados ao mesmo tempo (break externo;).
- Pegadinha for...in vs for...of (JS): for...in percorre as chaves/índices (como strings, gerando bugs de concatenação), enquanto for...of percorre os valores.
 - Bug do Zune (2008): Um laço while sem avanço interno num ano bissexto travou os aparelhos todos por entrar em ciclo infinito.

Desvio Incondicional (goto): Famoso pelo artigo de Dijkstra (1968) "Go To Statement Considered Harmful". O famoso bug da Apple (2014) aconteceu por um goto fail; duplicado que saltava a verificação da assinatura do certificado de segurança.

Comandos Protegidos de Dijkstra: Se múltiplas guardas forem verdadeiras, a escolha é não-determinística (aleatória). Este conceito é a base do canal select em Go, que sorteia uma das opções quando vários canais estão prontos ao mesmo tempo.

MÓDULO 6

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 - JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 - Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

1 - JavaScript

A saída é “0012”, o “for ... in” percorre as propriedades dos índices , que são fornecidos como Strings, para corrigir, utilizamos “for ... of”

2 - Python

A saída é “180.0”, a regra que causa o problema é “d = d or 10”, assume “10” como verdadeiro e atribui 10 de desconto à variável ‘d’, para corrigir removemos o “or 10”, deixando “d = d”

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

3 - Java

A saída é “Inválido”, o que causa esse problema é a ausência de “breaks”, a sua inclusão já resolve.

4 - C

A saída é “Saldo insuficiente fim”, esse problema é causado pelo ponto e vírgula após o if, fazendo com que o printf após o if execute tanto no caso verdadeiro quanto no caso falso.

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

5 - Go

A saída é: “dentro: 20 fora: 10”, o problema encontrado é que no primeiro print, estamos utilizando um novo X, declarado somente dentro do if, então ao sairmos do if, o seu valor retorna de 20 para 10 novamente, para corrigir, trocamos o “:=” para “=” dentro do if.

6 - Java

A saída é 0.0%, isso é ocasionado porque estamos tentando transformar um inteiro em um real, para corrigirmos podemos fazer um cast em double.